

75. Sort Colors Medium

Acceptance: 68.9% | [LeetCode](#)

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
**Input:** nums = [2,0,2,1,1,0]
**Output:** [0,0,1,1,2,2]
```

Example 2:

```
**Input:** nums = [2,0,1]
**Output:** [0,1,2]
```

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either `0`, `1`, or `2`.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

Claude Opus Solution

Sort Colors — The Dutch National Flag Problem

1. Intuition

The key insight is that we only have **three distinct values** (0, 1, 2). This is a special sorting scenario that doesn't require a general-purpose $O(n \log n)$ sort.

The classic pattern here is the **Dutch National Flag algorithm** (named by Edsger Dijkstra, referencing the three-colored flag). The core idea:

Partition the array into three regions in a single pass using three pointers. As we scan, we "throw" 0s to the left region and 2s to the right region, leaving 1s naturally in the middle.

Think of it like organizing a bookshelf where you only push small items left and big items right; the medium ones settle in the middle automatically.

2. Approach

We maintain three pointers that define **four regions** of the array:

```
[0 ... left-1] → all 0s   (sorted)
[left ... mid-1] → all 1s  (sorted)
[mid ... right] → UNKNOWN (unprocessed)
[right+1 ... n-1] → all 2s (sorted)
```

Pseudocode:

```
left = 0, mid = 0, right = n-1
while mid <= right:
    if nums[mid] == 0:
        swap(left, mid) # send 0 to left region
        left++, mid++   # both safe to advance (see below)
    elif nums[mid] == 2:
        swap(right, mid) # send 2 to right region
        right--          # DON'T advance mid (new value unknown)
    else: # nums[mid] == 1
        mid++           # 1 belongs in middle, just move on
```

Critical detail — why `mid++` after swapping a 0 but NOT after swapping a 2: - When we swap with `left`, the element coming from `left` was already processed (it's a 1, since `left ≤ mid`). Safe to advance `mid`. - When we swap with `right`, the element coming from `right` is **unprocessed**. We must re-examine it, so we do NOT advance `mid`.

3. Python Solution

```
from typing import List

class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Dutch National Flag algorithm – single pass, in-place.
        Maintains three regions: 0s | 1s | unknown | 2s
        """
        # left : boundary; everything before it is 0
        # mid : current element being examined
        # right : boundary; everything after it is 2
        left, mid, right = 0, 0, len(nums) - 1

        # Process until the unknown region is exhausted.
        # Note: condition is `mid <= right`, NOT `mid < right`,
        # because element at `right` is still unprocessed.
        while mid <= right:

            if nums[mid] == 0:
                # Found a 0 – swap it into the left (0s) region.
                # The element we receive from `left` is guaranteed
                # to be a 1 (already processed), so advance both pointers.
                nums[left], nums[mid] = nums[mid], nums[left]
                left += 1
                mid += 1

            elif nums[mid] == 2:
                # Found a 2 – swap it into the right (2s) region.
                # The element we receive from `right` is UNPROCESSED,
                # so we must re-examine it: shrink `right` but keep `mid`.
                nums[right], nums[mid] = nums[mid], nums[right]
                right -= 1

            else: # nums[mid] == 1
                # A 1 belongs in the middle region – just move forward.
                mid += 1
```

4. Complexity Analysis

Metric	Complexity	Explanation
Time	O(n)	<code>mid</code> and <code>right</code> together move toward each other; each element is examined at most once, satisfying the one-pass follow-up.
Space	O(1)	Only three integer pointers are used; sorting is done in-place.

This beats a comparison-based sort's $O(n \log n)$ lower bound because we exploit the constraint that values are limited to $\{0, 1, 2\}$.

5. Alternative Approaches

A) Counting Sort (Two-Pass)

Count occurrences of each value, then overwrite the array.

```
def sortColors(self, nums: List[int]) -> None:
    count = [0, 0, 0]
    for x in nums: # Pass 1: tally counts
        count[x] += 1
    i = 0
    for color in range(3): # Pass 2: rewrite array
        for _ in range(count[color]):
            nums[i] = color
            i += 1
```

- **Time:** $O(n)$, **Space:** $O(1)$ (fixed 3-element count array)
- Simpler to reason about, but requires **two passes** (doesn't satisfy the one-pass follow-up).

B) Generic Sort

`nums.sort()` — $O(n \log n)$, but the problem explicitly **forbids** the library sort.

Key Takeaways

- **Pattern:** Three-way partitioning (Dutch National Flag) — useful whenever you partition into ≤ 3 categories around pivot(s), e.g., quicksort's 3-way partition for duplicate-heavy arrays.
- **Trap to remember:** Don't advance `mid` after swapping with `right` — the incoming element is unverified.
- **Loop boundary:** Use `mid <= right` since `right`'s position is still part of the unknown region.

Codex Solution

1. Intuition

This is the classic **Dutch National Flag** problem.

We only have three possible values:

- `0` = red
- `1` = white
- `2` = blue

The goal is to rearrange them in this order:

`0`s first, then `1`s, then `2`s

The key insight is to maintain three regions while scanning the array:

<code>[0 ... left - 1]</code>	<code>-></code> all <code>0</code> s
<code>[left ... mid - 1]</code>	<code>-></code> all <code>1</code> s
<code>[mid ... right]</code>	<code>-></code> unknown
<code>[right + 1 ... end]</code>	<code>-></code> all <code>2</code> s

We use three pointers:

- `left`: position where the next `0` should go
- `mid`: current element being inspected
- `right`: position where the next `2` should go

Depending on `nums[mid]`:

- If it is `0`, swap it with `nums[left]`
- If it is `1`, leave it and move on
- If it is `2`, swap it with `nums[right]`

This gives a **one-pass, in-place, constant-space** solution.

2. Approach

Pseudocode

```
left = 0
mid = 0
right = len(nums) - 1

while mid <= right:
    if nums[mid] == 0:
        swap nums[left] and nums[mid]
        left += 1
        mid += 1

    else if nums[mid] == 1:
        mid += 1

    else if nums[mid] == 2:
        swap nums[mid] and nums[right]
        right -= 1
```

Important detail

When `nums[mid] == 2`, we swap with `nums[right]`, but we do **not** increment `mid`.

Why?

Because the element swapped from the right side into `mid` is still unknown. It could be `0`, `1`, or `2`, so we need to inspect it in the next iteration.

3. Python Solution

```
from typing import List

class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Sort nums in-place so that all 0s come first,
        followed by all 1s, followed by all 2s.

        This uses the Dutch National Flag algorithm.

        The function modifies nums directly and returns None.
        """

        # left points to the position where the next 0 should be placed.
        left = 0

        # mid scans through the array.
        mid = 0

        # right points to the position where the next 2 should be placed.
        right = len(nums) - 1

        # Process elements while there is still an unknown region.
        while mid <= right:

            # Case 1: nums[mid] is 0.
            # It belongs in the left region.
            if nums[mid] == 0:
                # Swap current 0 into its correct region.
                nums[left], nums[mid] = nums[mid], nums[left]

                # The 0-region expands by one.
                left += 1

            # The element swapped into mid is safe to skip because:
            # - If left < mid, it must have been a 1 from the middle region.
            # - If left == mid, we swapped the element with itself.
            mid += 1

            # Case 2: nums[mid] is 1.
            # It is already in the correct middle region.
            elif nums[mid] == 1:
                mid += 1

            # Case 3: nums[mid] is 2.
            # It belongs in the right region.
            else:
                # Swap current 2 into its correct region.
                nums[mid], nums[right] = nums[right], nums[mid]

                # The 2-region expands by one from the right.
                right -= 1

            # Do NOT increment mid here.
            # The new nums[mid] came from the unknown region,
            # so we must inspect it in the next loop iteration.
```

4. Complexity Analysis

Time Complexity

$O(n)$

Each element is processed at most a constant number of times.

The `mid` pointer moves from left to right, and the `right` pointer moves from right to left. Therefore, the total number of operations is linear in the length of the array.

Space Complexity

$O(1)$

The algorithm only uses three pointers:

`left`, `mid`, `right`

No extra array or data structure is used.

5. Alternative Approaches

Approach 1: Counting Sort

Since there are only three possible values, we can count how many `0`s, `1`s, and `2`s appear.

Then overwrite the original array.

Example:

```
count0 = nums.count(0)
count1 = nums.count(1)
count2 = nums.count(2)

nums[:] = [0] * count0 + [1] * count1 + [2] * count2
```

However, this is less ideal because it may use extra space depending on implementation.

A more in-place version:

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        count = [0, 0, 0]

        for num in nums:
            count[num] += 1

        index = 0

        for color in range(3):
            for _ in range(count[color]):
                nums[index] = color
                index += 1
```

Complexities:

Time: $O(n)$
Space: $O(1)$

This is simple, but it requires two passes.

Approach 2: Built-in Sort

```
nums.sort()
```

This is not allowed by the problem statement.

Also, it is usually:

Time: $O(n \log n)$
Space: depends on sorting implementation

Key Pattern

This problem is a perfect example of:

Three-pointer partitioning

or more specifically:

Dutch National Flag algorithm

It is useful when you need to partition an array into three groups in-place.